

Architectural Pattern Selection & Justification: Microservice Catalog & Health Monitoring Portal (Problem #42)

1. Architectural Choice

We chose the **Microservices Architecture pattern** for the Microservice Catalog & Health Monitoring Portal.

2. Scenario-Related Reasons for Selection

- **Separate Scaling Needs for Ingestion and UI Rendering:** The **Health Prober Component** runs high-frequency automated polling (30-second ping cycles across hundreds of endpoints) and streams data straight to the **Time-Series Metric Store Component (FR-001, NFR-002)**. On the other hand, the **Graph & Docs Visualizer Component (FR-003, NFR-001)** handles occasional read-heavy user requests that require client-side processing to display over 200 nodes. Microservices allow the ingestion engine to scale horizontally with network probes without using up resources needed for rendering complex topologies.
- **Fault Isolation for Monitoring Reliability:**
The main non-functional requirement (**NFR-002**) requires 99.9% uptime for health checks and prevents data loss during service outages. Separating the **Health Prober** and **Alert Engine** from the web portal ensures that background probe loops continue logging metrics and sending alerts even if user queries or rendering crashes the UI.

3. Security Advantage

- **Least-Privilege Network and Credential Boundary Segregation:**
Microservices allow for clear network segmentation and API security boundaries. The **Alert Engine Component** needs write access to external webhooks (like Slack, PagerDuty, and SMTP) and stores communication credentials. In this architecture, those credentials stay isolated inside the alert container. The public catalog viewer never gets direct network routing or tokens for notification webhooks, which limits the potential damage if the frontend is compromised.

4. Performance Benefit

- **Independent Resource Allocation for Heavy Graph Rendering (< 2-Second Latency):**
Under **NFR-001**, the system must quickly render architecture graphs with up to 200 microservices without lag. In a monolith, complex graph traversals and OpenAPI parsing compete directly with real-time health checks for CPU time. With microservices, the **Graph & Docs Visualizer** handles topology rendering on dedicated instances or cached read replicas, leaving the health prober fast and responsive for automated checks.